Advanced Embedded Systems Lab

Final Documentation

Smart Greenhouse IoT Monitoring and Control System

Team A5 - Summer Term 2026

Date: 07 July 2026

Repository: https://github.com/sohan2961/SS2026_AES_Lab_Team_A5

Team Member	Main Responsibility	Documentation Sections
Md Mostafizur Rahman	Raspberry Pi central controller, MQTT broker, Flask dashboard, system integration	Concept, technologies, implementation, results
Turja Barua	ESP32 environment node, temperature/humidity sensing, threshold regulator integration	Introduction, concept, environment node, testing
Moaz Bin Alamgir Chowdhury	Arduino Uno WiFi Rev2 fire and gas safety node with MQ-2, KY-026, buzzer and RGB LED	Technologies, safety node implementation
Deepak Kapil	Arduino Uno WiFi Rev2 actuator node, KY-019 relay, MQTT relay control	Actuator node implementation, relay test results

1 Team Members

The prototype was developed as a team project. Each team member worked on one hardware/software subsystem and then integrated the subsystem through MQTT communication. The table below summarizes the team structure and the main contribution of each member.

Name	Subsystem	Hardware / Software	Main Output
Md Mostafizur Rahman	Central controller	Raspberry Pi 3, Mosquitto MQTT, Python Flask, CSV logging, systemd service	Live web dashboard, MQTT broker, storage of sensor data and central relay decision logic
Turja Barua	Environment monitoring node	ESP32, KY-015 temperature/humidity sensor, rotation sensor/regulator, I2C LCD	Publishes temperature, humidity and selected threshold temperature to the Raspberry Pi
Moaz Bin Alamgir Chowdhury	Fire and gas safety node	Arduino Uno WiFi Rev2, MQ-2 gas/smoke sensor, KY-026 flame sensor, buzzer, RGB LED	Detects safety risk and publishes alarm/status messages to the central controller
Deepak Kapil	Actuator/relay node	Arduino Uno WiFi Rev2, KY-019 relay module, optional external LED actuator simulation	Receives MQTT ON/OFF commands and switches the relay for fan/pump simulation

2 Introduction

In this project, Internet of Things (IoT) means that physical greenhouse components such as sensors, controllers and actuators are connected over a wireless network so that they can exchange data automatically. A Wireless Sensor Network (WSN) means that distributed nodes collect environmental values and send them to a central point without wired data communication.

The target application is a small smart greenhouse prototype. The greenhouse should monitor important environmental and safety values, show them on a dashboard and react automatically when the values cross a user-defined limit. The prototype is not intended as a commercial greenhouse controller; it is a laboratory demonstrator for embedded networking, MQTT messaging, sensor integration and actuator control.

The most important new control function is automatic relay control using a regulator threshold. The user can change the threshold temperature with a rotation sensor. The ESP32 publishes this threshold to the Raspberry Pi. The Raspberry Pi compares the current temperature with the threshold and sends an ON or OFF command to the Arduino relay node.

Example behavior: if the current temperature is 31 C and the regulator threshold is 28 C, the relay turns ON. If the threshold is later increased to 35 C, the relay turns OFF again. This makes the relay behavior dynamic and user-controlled instead of using a fixed hard-coded temperature.

3 Concept Description

The main application of the prototype is greenhouse monitoring and automatic actuator control. The system is split into small nodes. Each node has a clear responsibility and communicates through MQTT topics over Wi-Fi. The Raspberry Pi is the central controller and MQTT broker. The ESP32 boards publish sensor values. The Arduino Uno WiFi Rev2 receives commands and switches the relay actuator.

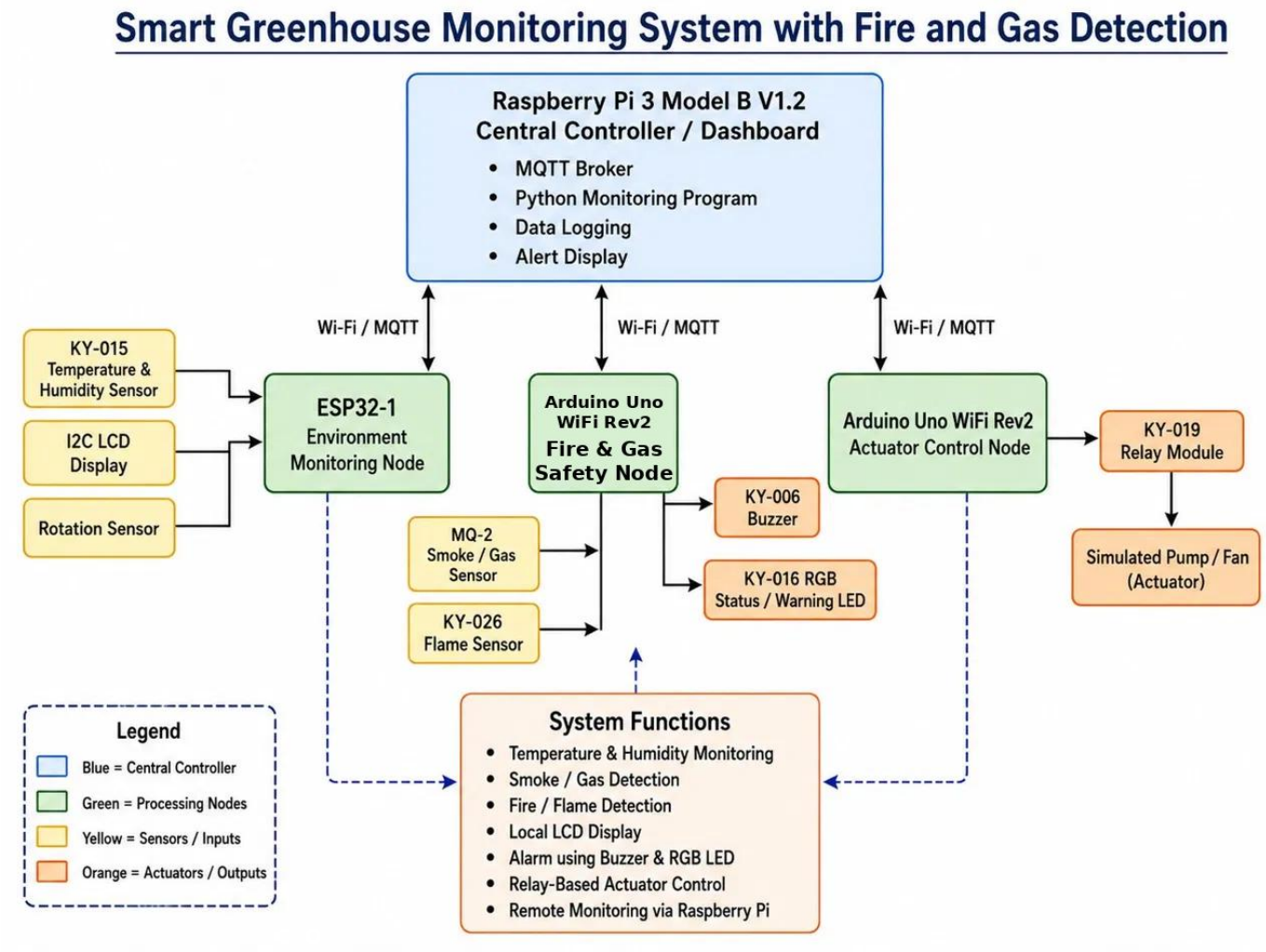

Figure 1: Target application block diagram of the Smart Greenhouse prototype.

3.1 Devices, Sensors and Actuators

Device / Module	Project Use	Interface / Communication
Raspberry Pi 3 Model B V1.2	Central controller, MQTT broker, dashboard server and data logger	Wi-Fi/Ethernet, Python, Flask, Mosquitto, Paho MQTT
ESP32 environment node	Reads temperature, humidity and threshold regulator value	Wi-Fi MQTT, digital DHT interface, analog rotation sensor input
Arduino Uno WiFi Rev2 safety node	Detects gas/smoke and flame risk and triggers warning outputs	Wi-Fi MQTT, analog/digital sensor inputs, GPIO outputs
Arduino Uno WiFi Rev2	Controls KY-019 relay from MQTT command	Wi-FiNINA, PubSubClient, GPIO

		D7
KY-015 temperature/humidity sensor	Measures greenhouse air condition	Digital data pin
Rotation sensor / regulator	User-defined threshold temperature selection	Analog input mapped to threshold range
MQ-2 and KY-026	Gas/smoke and flame detection	Analog/digital inputs
KY-019 relay	Actuator switching for fan/pump simulation	Relay signal pin with external load on COM/NO contacts

3.2 MQTT Topic Plan

The nodes communicate through publish/subscribe topics. The most important correction in the current implementation is that relay commands are now sent to the actuator topic instead of the old pump topic.

MQTT Topic	Direction	Meaning
greenhouse/temperature	ESP32 -> Raspberry Pi	Current greenhouse temperature in degrees Celsius
greenhouse/humidity	ESP32 -> Raspberry Pi	Current relative humidity value
greenhouse/rotation	ESP32 -> Raspberry Pi	Threshold temperature selected by the regulator/rotation sensor
greenhouse/smoke	Arduino Uno WiFi Rev2 -> Raspberry Pi	Gas or smoke sensor value
greenhouse/flame	Arduino Uno WiFi Rev2 -> Raspberry Pi	Flame detection state or value
greenhouse/alarm	Arduino Uno WiFi Rev2 -> Raspberry Pi	Alarm status from safety node
greenhouse/actuator/relay/set	Raspberry Pi -> Arduino	ON/OFF command for relay actuator
greenhouse/actuator/relay/status	Arduino -> Raspberry Pi	Relay node connection and relay state feedback

4 Project / Team Management

The project was managed with a modular team approach. Each person built and tested one node independently, then all nodes were integrated using common MQTT topics. This reduced dependency between team members and made debugging easier because every module could be tested separately with mosquitto_pub and mosquitto_sub commands before final integration.

- GitHub was used for version control and sharing project files.
- The work was divided by hardware node: Raspberry Pi, ESP32 environment, Arduino Uno WiFi Rev2 safety and Arduino relay.
- MQTT topics were defined as the integration contract between modules.
- System integration was tested from the terminal first, then through the web dashboard.
- The running Raspberry Pi dashboard file was copied back into the GitHub project folder before committing and pushing.

Task	Method Used	Reason
Node development	Parallel work by subsystem	Each team member could test hardware and software independently
Integration	MQTT topic-based interface	Loose coupling between different microcontrollers
Testing	Command-line MQTT tests and web dashboard tests	Fast debugging and clear proof of communication
Version control	Git commit and push to main branch	Traceability of changes and final code availability
Service deployment	systemd service greenhouse.service	Dashboard starts automatically and can be restarted reliably

During integration, one important issue was discovered: the dashboard originally sent manual commands to greenhouse/pump, but the Arduino relay node subscribed to greenhouse/actuator/relay/set. This mismatch meant the dashboard command could not reach the actuator. The topic was corrected in app.py, and the GitHub repository was updated after the fix.

5 Technologies

5.1 Sensors and Actuators

The prototype uses low-cost educational sensor modules. The environment node uses the KY-015 sensor to measure temperature and humidity. A rotation sensor is used as a manual regulator so the user can change the temperature threshold. The safety node uses an MQ-2 sensor for smoke/gas detection and a KY-026 flame sensor for fire indication. The actuator node uses a KY-019 relay module to switch a load such as a small fan, pump or external LED simulation.

The relay module required special attention because many relay modules are active LOW. In that case, writing LOW to the signal pin activates the relay, and writing HIGH deactivates it. The Arduino code was adjusted so that the serial status LED and the physical relay behavior match the expected ON/OFF command.

5.2 Communication Protocols

MQTT is used as the main communication protocol because it is lightweight, simple and suitable for sensor networks. Each node either publishes sensor data or subscribes to a command topic. The Raspberry Pi runs the Mosquitto broker. The dashboard is implemented with Flask and provides an HTTP web interface for users.

- Wi-Fi connects ESP32, Arduino Uno WiFi Rev2, Raspberry Pi and laptop/phone browser.
- MQTT publish/subscribe is used for all sensor data and actuator commands.
- HTTP is used between the browser and the Flask dashboard.
- CSV logging stores timestamped sensor values and relay commands for later review.

5.3 Programming Languages and Libraries

Part	Language / Library	Purpose
Raspberry Pi dashboard	Python, Flask, Paho MQTT, CSV, threading	Web dashboard, MQTT subscriber, relay decision logic and logging
ESP32 nodes	Arduino C/C++, WiFi library, PubSubClient, DHT library	Sensor reading and MQTT publishing
Arduino relay node	Arduino C/C++, WiFiNINA, PubSubClient	Receive MQTT command and switch relay pin
Deployment	systemd, Git, GitHub	Run dashboard as service and manage project updates

6 Implementation

The implementation is organized as a distributed embedded system. The central module is the Raspberry Pi application app.py. It subscribes to all sensor topics, stores the latest values in a shared state dictionary, updates history buffers, writes CSV log entries and serves the web dashboard. It also decides the automatic relay command when temperature or threshold values change.

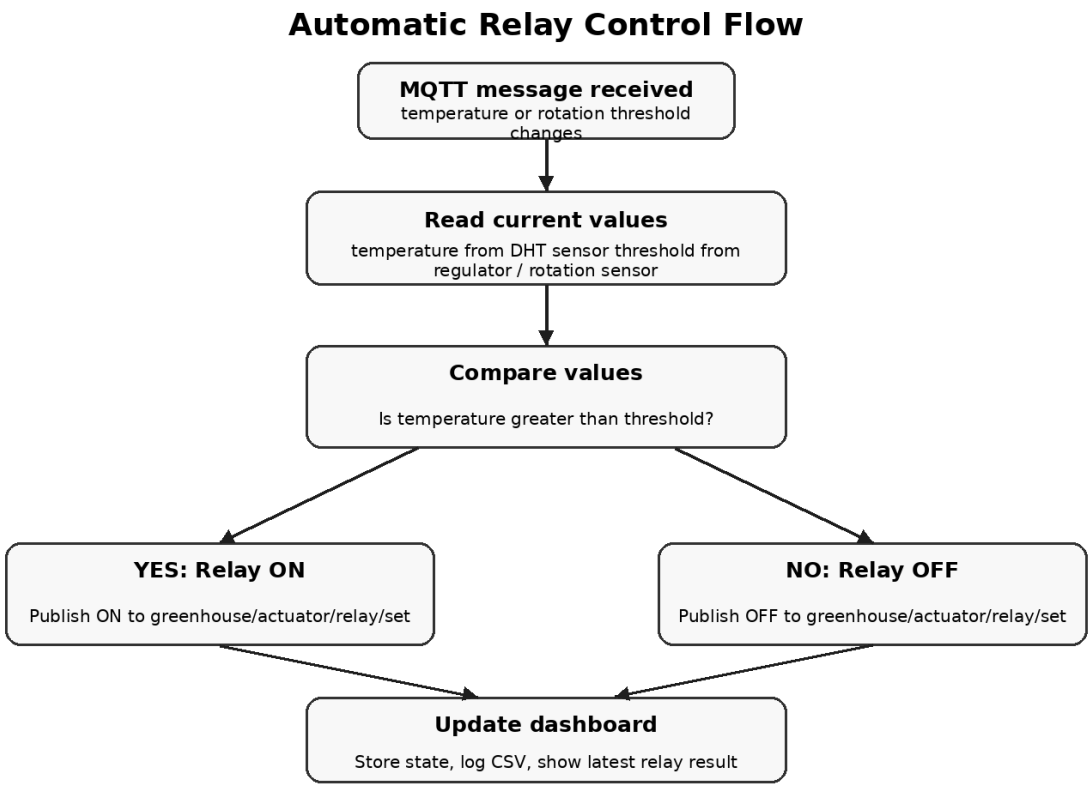

Figure 2: Automatic relay control flow implemented in the Raspberry Pi app.py.

6.1 Raspberry Pi Central Controller

The Raspberry Pi has three main roles. First, it runs the Mosquitto MQTT broker so that all devices can communicate. Second, it runs the Flask dashboard that displays live values to the user. Third, it makes the automatic control decision for the relay based on the current temperature and the regulator threshold.

Important modules/functions in app.py:

- TOPICS list: defines all MQTT topics that the dashboard should subscribe to.
- state dictionary: stores the latest temperature, humidity, smoke, flame, rotation threshold, alarm and relay status.
- on_message(): receives MQTT values, updates state, writes logs and triggers automatic relay control.
- auto_control_relay(): compares temperature and threshold and publishes ON/OFF to the Arduino relay topic.
- api_pump(): keeps compatibility with the dashboard button but now publishes to greenhouse/actuator/relay/set.
- greenhouse.service: runs /home/sohan/rpi_central/app.py automatically using the virtual environment.

6.2 Automatic Relay Algorithm

The threshold is not fixed in the program. It is controlled by the regulator/rotation sensor. The ESP32 maps the analog regulator value to a temperature threshold and publishes it using the greenhouse/rotation topic. The Raspberry Pi then compares this threshold with the latest temperature.

```
if temperature > threshold:
    command = "ON"
else:
    command = "OFF"
```

```
mqtt_client.publish("greenhouse/actuator/relay/set", command)
```

To avoid sending the same command continuously, the program stores the last relay command. A new MQTT command is published only when the desired relay state changes.

6.3 Environment Node

The environment node uses an ESP32 to read the current temperature, humidity and regulator value. The regulator value is used to select the threshold temperature. A typical mapping can convert the analog input range 0-4095 into a practical threshold range, for example 20 C to 40 C. The ESP32 then publishes temperature, humidity and threshold as MQTT messages.

Published Value	MQTT Topic	Example
Temperature	greenhouse/temperature	31
Humidity	greenhouse/humidity	57
Threshold from regulator	greenhouse/rotation	28

6.4 Safety Node

The safety node reads gas/smoke and flame data. When a dangerous situation is detected, the node can switch local warning outputs such as buzzer and RGB LED and also publish alarm information to the Raspberry Pi. This separates safety monitoring from the environment node and keeps the design modular.

The safety node is a modular part of the system that constantly checks the greenhouse for hazards. We kept the safety monitoring separate from the regular environment monitoring so it can react instantly. If there is a danger, it triggers the buzzer and warning lights right away. In this version, we used an Arduino Uno WiFi Rev2. The Arduino handles reading the sensors, deciding if there is an emergency, and sending the data over MQTT.

For the hardware, the node uses an MQ-2 sensor to detect gas and smoke, along with a KY-026 flame sensor to watch for the infrared light made by open fires. For local warnings in the room, it uses a KY-006 buzzer and an LED light. The node also sends all of its data straight to the Raspberry Pi using three specific MQTT topics:

Published Value	MQTT Topic	Safety Node Interpretation
Gas / Smoke Value	greenhouse/smoke	Raw sensor value for gas or smoke concentration
Flame Detection	greenhouse/flame	Current flame detection state or raw analog value
Alarm Status	greenhouse/alarm	Evaluated safety status message sent to the central controller

We can tell the Arduino Rev2 is working and connected properly via MQTT because its data shows up perfectly in the central CSV log. A snippet from the CSV file shows that, the safety node sending live updates to the Raspberry Pi right alongside the environment node. The data shows that, the Arduino reporting the smoke level, a flame detection state, and successfully sending an active alarm state to the central dashboard:

Timestamp	Topic	Value
2026-06-10 13:56:11	greenhouse/temperature	24.5
2026-06-10 13:56:11	greenhouse/humidity	62.0
2026-06-10 13:56:11	greenhouse/smoke	350
2026-06-10 13:56:12	greenhouse/flame	1

2026-06-10 13:56:12	greenhouse/rotation	30
2026-06-10 13:56:12	greenhouse/alarm	1

6.5 Arduino Relay Node

The Arduino Uno WiFi Rev2 connects to the Raspberry Pi MQTT broker at IP address 172.20.10.4. It subscribes to greenhouse/actuator/relay/set and reacts to ON/OFF messages. The KY-019 relay signal pin is connected to Arduino digital pin D7. The Arduino publishes relay status on greenhouse/actuator/relay/status.

Final relay behavior after debugging:

- MQTT message ON -> setRelay(true) -> relay signal LOW for active LOW module -> physical relay ON.
- MQTT message OFF -> setRelay(false) -> relay signal HIGH for active LOW module -> physical relay OFF.
- Arduino built-in status LED is still used as a visible software state indicator.

7 Results

The final integrated prototype shows that the greenhouse nodes can communicate through the Raspberry Pi MQTT broker and that the relay can be controlled manually and automatically. The key result is dynamic threshold-based relay control: the regulator controls the threshold, and the relay state changes when the measured temperature crosses this threshold.

7.1 Integration Tests

Test Case	Command / Condition	Expected Result	Result
Dashboard service status	sudo systemctl status greenhouse.service	Flask dashboard runs as a systemd service	Service runs after syntax/indentation error was fixed
Manual relay topic test	mosquitto_pub -t greenhouse/actuator/relay/set -m "ON"	Arduino receives ON and switches relay state	Passed
Manual relay OFF test	mosquitto_pub -t greenhouse/actuator/relay/set -m "OFF"	Arduino receives OFF and switches relay state	Passed
Below threshold	rotation=28, temperature=25	Raspberry Pi publishes OFF	Passed
Above threshold	rotation=28, temperature=31	Raspberry Pi publishes ON	Passed
Threshold increased	rotation=35 while temperature=31	Raspberry Pi publishes OFF	Passed
GitHub update	git push origin main	Updated app.py is uploaded to GitHub main branch	Passed

7.2 Problems Faced and Solutions

Problem	Reason	Solution
Dashboard button did not control relay	Dashboard published to greenhouse/pump, while Arduino subscribed to greenhouse/actuator/relay/set	Changed the publish topic in app.py and updated the log topic
Service failed after editing app.py	Python indentation error after the if statement that calls auto_control_relay()	Fixed indentation and checked journalctl output
Relay LED/status worked but relay stayed off	Relay module behavior was active LOW, not active HIGH	Inverted relay pin logic: LOW = relay ON, HIGH = relay OFF
Git push rejected	Remote GitHub branch had newer work	Used git pull --rebase origin main and then pushed again

7.3 Final Demonstration Scenario

1. Start Raspberry Pi and confirm greenhouse.service is active.
2. Open the dashboard in a browser at http://172.20.10.4:5000.
3. Connect ESP32 environment node and publish temperature/humidity/rotation threshold values.
4. Connect Arduino relay node and confirm it subscribes to greenhouse/actuator/relay/set.
5. Rotate the regulator to select a threshold temperature.
6. Increase the temperature value above the threshold and observe the relay turning ON.
7. Increase the threshold above the current temperature and observe the relay turning OFF.

The demonstration proves that the prototype can react automatically to environmental data and that the threshold can be changed by the user through the regulator. This is the main functional requirement for the actuator control part of the smart greenhouse.

7.4 Limitations and Future Improvements

- Sensor calibration should be improved before using the system for real plants.
- A hysteresis range could be added so the relay does not switch too frequently around the threshold.

- The threshold value could be stored persistently so it is not lost after restart.
- The dashboard could add a visible auto/manual mode switch and show the selected threshold more clearly.
- MQTT authentication and Wi-Fi security should be added for a more realistic deployment.

8 Sources / References

Project repository: https://github.com/sohan2961/SS2026_AES_Lab_Team_A5

- Raspberry Pi Documentation: https://www.raspberrypi.com/documentation/
- Eclipse Mosquitto MQTT Broker: https://mosquitto.org/documentation/
- Flask Documentation: https://flask.palletsprojects.com/
- Eclipse Paho MQTT Python Client: https://eclipse.dev/paho/
- Arduino Uno WiFi Rev2 Documentation: https://docs.arduino.cc/hardware/uno-wifi-rev2/
- Arduino WiFiNINA Library: https://docs.arduino.cc/libraries/wifinina/
- Arduino PubSubClient Library: https://pubsubclient.knolleary.net/
- Team source code and test logs from the Smart Greenhouse prototype.

Appendix A: Useful Test Commands

The following commands were used during debugging and demonstration:

```
sudo systemctl restart greenhouse.service
sudo systemctl status greenhouse.service
sudo journalctl -u greenhouse.service -n 50 --no-pager
mosquitto_sub -t greenhouse/actuator/relay/set -v
mosquitto_pub -t greenhouse/rotation -m "28"
mosquitto_pub -t greenhouse/temperature -m "31"
git pull --rebase origin main
git push origin main
```